

计算机系统结构实验报告Lab02: 4-bit adder

uu-matter543

2024-2025-2

摘要

在 Lab02 中，我成功搭建了 1 位加法器；学习了如何组合模块以实现更高层的模块，并尝试添加已有的设计模块；并基于 1 位加法器搭建了 4 位加法器，进行了上板验证，收获颇丰。

目录

1	实验目的	2
2	实验原理	2
3	功能实现	2
3.1	1 位加法器	2
3.2	4 位加法器	3
4	功能仿真	3
5	上板验证	4
5.1	修改工程文件	4
5.2	上板验证	4
6	总结与反思	4

1 实验目的

1. 掌握 Xilinx 逻辑设计工具 Vivado 的基本操作
2. 掌握 Verilog HDL 进行简单的逻辑设计
3. 使用功能仿真
4. 约束文件的使用和直接写法
5. 生成 Bitstream 文件
6. 上板验证

2 实验原理

四位全加器可由四个一位全加器级联而成，而一位全加器可由两个半加器组合而成，也可用基本的逻辑门实现。本实验采用逻辑门实现。

3 功能实现

3.1 1 位加法器

```
module adder_1bit(  
    input a,  
    input b,  
    input ci,  
    output s,  
    output co  
);  
    wire s1, c1, c2, c3;  
    and (c1, a, b), (c2, b, ci), (c3, a, ci);  
    xor (s1, a, b), (s, s1, ci);  
    or (co, c1, c2, c3);  
endmodule
```

定义 a, b, ci 分别作为操作数 1，操作数 2，输入进位；定义 s, co 分别为输出和，输出进位；定义 s1, c1, c2, c3 为中间变量，供以下逻辑模块使用。

根据组合逻辑，输出和为 1 当且仅当 (a, b, ci) 三个操作数里有 1 或 3 个 1 时，可用两级异或逻辑实现；输出进位为 1 当且仅当 (a, b, ci) 三个操作数中有 2 或 3 个 1 时，可用两级与或组合逻辑实现。

3.2 4 位加法器

```

module adder_4bits(
    input [3:0] a,
    input [3:0] b,
    input ci,
    output [3:0] s,
    output co
);
wire [2:0] ct;
adder_1bit a1(.a(a[0]), .b(b[0]), .ci(ci), .s(s[0]), .co(ct[0])),
    a2(.a(a[1]), .b(b[1]), .ci(ct[0]), .s(s[1]), .co(ct[1])),
    a3(.a(a[2]), .b(b[2]), .ci(ct[1]), .s(s[2]), .co(ct[2])),
    a4(.a(a[3]), .b(b[3]), .ci(ct[2]), .s(s[3]), .co(co));
endmodule

```

只需要组合 4 个 1 位全加器即可，要注意需要保存级联进位为中间变量，以便于在 4 级 1 位全加器传递；同时应注意根据数字逻辑填写数组下标。

4 功能仿真

根据实验指导书编写激励文件，只需要在不同时刻改变输入信号就可验证逻辑正确性。仿真图如下：

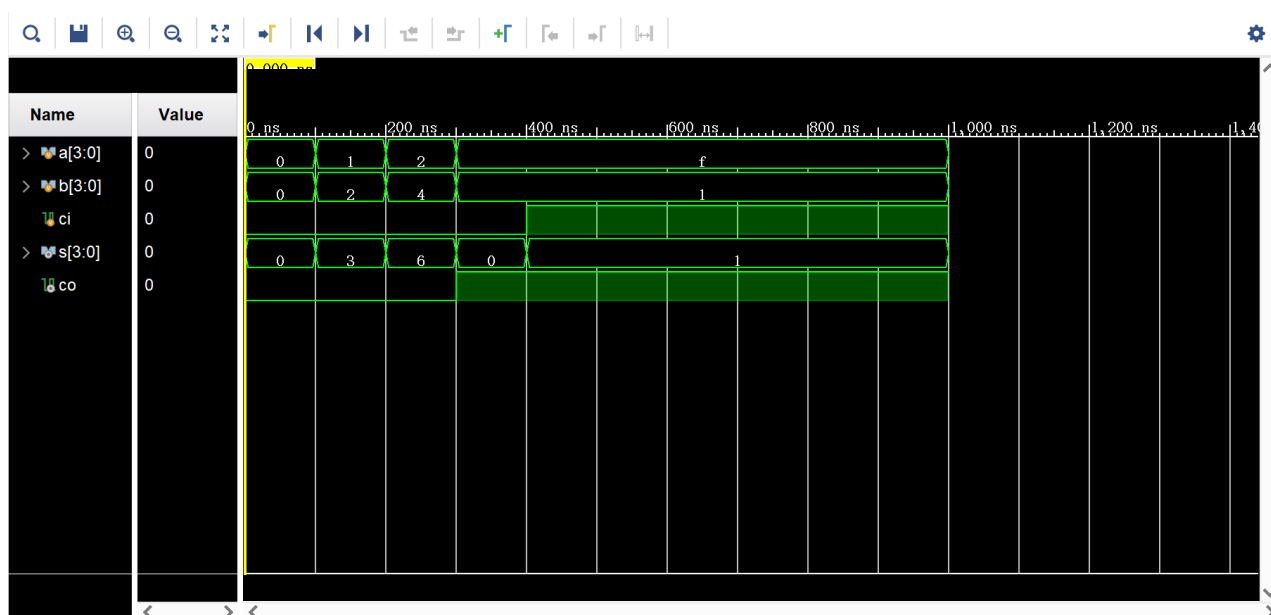


图 1: 4 位全加器仿真效果图

可以观察到, $0 + 0 = 0$, $1 + 2 = 3$, $2 + 4 = 6$, $0fh + 1h = 10h$, $0fh + 1h + 1h = 11h$, 均符合加法器逻辑。

5 上板验证

5.1 修改工程文件

创建 Top.v 文件, 将实现细节隐藏, 将全加器与 LED、7 段数码管连接, 并添加时钟以提供触发沿; 并将提供的 IP 添加进工程文件即可。

5.2 上板验证

根据实验指导书连接设备, 将程序生成二进制流文件, 并写入开发板, 可以在开发板上观察到两个输入操作数的和; 调整输入操作数, 能观察到开发板的数码管上显示正确的和, 说明加法器功能已经实现。

6 总结与反思

在 Lab01 的基础上, 我对 Vivado IDE 的操作变得更加熟练, 完成实验的速度也比 Lab01 快了很多。在本次实验中我成功实现了 4 位加法器, 也熟悉了通过低层逻辑模块组合形成高层逻辑模块的设计思路 and 开发流程。感谢老师和助教的指导, 也感谢课程组用心制作的实验指导书。